

CS 236: Operating Systems Lab

Spring 2025-26

Lab 3: — — — ● ● ●

1. pipes are for plumbing. plumbing is a life skill.

Refer to the manpage `pipe(2)` for a description of the `pipe` system call.

A basic C program template for parts a, b, and c is already provided in the lab. You are expected to complete the required logic using this template.

- a. Write a program `p1a.c` that creates a pipe using `pipe()`, writes a message into the write-end of the pipe, and then reads that message back from the read-end to display it.

Sample output

```
Shell
> ./p1a
Write to pipe: Hello World
Read from pipe: Hello World
```

- b. Write a program `p1b.c` that demonstrates bidirectional communication between a parent and a child process using pipes. The program should create two pipes: one for communication from the parent to the child and another for communication from the child to the parent. The parent process should send the message “Ping” to the child through the first pipe. The child should read and print this message, then send the message “Pong” back to the parent through the second pipe. Finally, the parent should read and print the received “Pong” message to the console.

Sample output

```
Shell
> ./p1b
[PID=66480, PPID=66479] Child received: Ping
[PID=66479, PPID=65527] Parent received: Pong
```

- c. Write a program in `p1c.c` that demonstrates interprocess communication among three processes created using one parent process and two child processes. The parent process should create two child processes using `fork()` and use pipes for

communication between them. A total of three pipes should be used to enable one-way communication between each pair as described below.

- The **parent process (Process 1)** should take an integer value as input and pass it to the first Child Process (**Process 2**) through the first pipe.
- The **first Child Process (Process 2)** should read and print the received number, then multiply it by **2** and send the updated number to the second **Child Process (Process 3)** via the second pipe.
- The **second Child Process (Process 3)** should read and print the received number, then multiply it by **2** again and send the final result back to the **parent process (Process 1)** through the third pipe.
- Finally, the parent process should read and print the number received from the second Child Process.

Sample output

```
Shell
> ./p1c
Enter a Number: 10
Child 1 received: 10
Child 2 received: 20
Parent received: 40
```

2. to signal is to communicate

Refer to the manpages [signal\(7\)](#), [sigaction\(2\)](#) for details of the system calls.

- Write a program `p2a.c` that loops indefinitely. When it receives a SIGINT signal (via `Ctrl-C`), It should print the message:
`"You can't interrupt me :), PID: <PID of the process>"`
and continue. You need to set a signal handler for SIGINT.
Note: `^C` stands for Ctrl-C in the terminal; you need not print it.

How would you stop this process from executing?

Hint: What are other ways to deliver signals to a process?

Sample output

```
Shell
> gcc p2a.c -o p2a
> ./p2a
^CYou can't interrupt me :), PID: 319267
^CYou can't interrupt me :), PID: 319267
```

- b. Write a program `p2b.c` that forks a single child process. The child process should wait for 3 seconds before performing an `exit`. The parent process should reap it without polling or blocking on wait. A process sends the SIGCHLD signal to its parent on exit. You need to write a signal handler that reaps the child process and prints its PID upon receiving the signal.
- c. Extend `p2b.c` to `p2c.c` so it behaves as a basic version of the `time` command. It should take a simple shell command as an argument, fork a child process to `exec` that command, then print the user time consumed, the system time consumed by the child process, and the child's exit code. You can use the `siginfo_t` structure to get this info in the signal handler. Check your output with the actual `time` command provided by your shell.

The command in the sample output is for reference; you can use any long-running program to test your program.

Note: User time is CPU time spent executing a process's instructions. System time is the CPU time the OS spends running on the process's behalf, e.g., when the process performs a syscall.

Sample output

```
Shell
> ./p2c awk 'BEGIN{for(i=0;i<1e8;i++);}';
Command: awk BEGIN{for(i=0;i<1e8;i++);}
System time: 0.000000, User time: 4.820000
Exit code: 0
> time awk 'BEGIN{for(i=0;i<1e8;i++);}';
awk 'BEGIN{for(i=0;i<1e8;i++);}' 4.84s user 0.00s system 99% cpu 4.847 total
```

- d. Modify `p2a.c` so that the process forks a child process and prints its PID. Name the program as `p2d.c`. If you now press Ctrl-C, you'd observe that both processes are printing the message, which means both are receiving the SIGINT signal. This is because they belong to the same process group, and the shell sends a SIGINT signal to every process in the foreground process group. Read manpage `setpgid(2)` to understand more about process groups. Modify your code so the child process no longer receives the SIGINT signal.

Hint: In the child process, you can call `setpgid(0, 0)` to place it in a new process group.

- e. Modify `p2b.c` so that the main program forks 2 child processes and then enters an infinite loop. Both children must print their PIDs, then sleep for 2 seconds, and then exit. The parent must sleep for 5 seconds before starting an infinite loop. Run this program multiple times. Is the program able to reap both the children always? Explain. Use the `ps` command in another terminal to verify that both children are reaped correctly.

Hint: If both children exit around the same time, the parent may not be notified separately for each termination because standard signals are not queued. Keep this behavior in mind.

3. shell done!

- a. Write a program named `p3a.c` that internally uses the existing UNIX `cat` command but modifies file descriptors to control its input and output behavior. The program should accept two command-line arguments: an input file and an output file, and should be executed as:

```
> ./p3a input.txt output.txt
```

The program must perform the following steps:

1. Open `input.txt` for reading and redirect the standard input (`STDIN`) of the process to this file using file descriptor manipulation (`open()`, etc.).
2. Open `output.txt` for writing (create it if it does not exist, truncate it if it does) and redirect the standard output (`STDOUT`) of the process to this file.
3. After redirecting the file descriptors, execute the inbuilt `cat` command using an appropriate `exec()` system call so that:
 - `cat` reads its input from `input.txt`
 - `cat` writes its output to `output.txt`

Restrictions

- The program **must not** use `read()` or `write()` for file I/O.
- The program **must not** execute `cat` with filenames as arguments (for example, **do not do** `exec cat input.txt output.txt`).
- All input and output redirection must be achieved **only through file descriptor manipulation**.

Hint

- `open()` always returns the **lowest-numbered free file descriptor**.

- Typically, **STDIN = 0** and **STDOUT = 1**, so redirecting input/output correctly allows `cat` to work without knowing the filenames.

b. Let's start building a shell now!

A tokenizer program, `shell.c` is provided as part of the lab. which will be helpful for the following tasks. You can start from the simple shell built in the last lab. For this task, the shell should handle simple commands that don't involve features such as redirection, piping, or background execution. You can assume an upper limit on the size of each command, the number of tokens it has, etc., but please define it as a macro rather than hardcode it.

The shell must gracefully handle all errors; i.e., it must not crash when given an empty/incorrect command; it should just notify and move on to the next prompt. You can rely on the `exec` command for this; it'll fail if the command does not exist. For other cases, the command itself would give the error. In any case, make sure this child process is reaped and move on to the next prompt. Please check that the child process is reaped via the `ps` command.

Note: All the following sub-tasks must be implemented by **incrementally modifying the same `shell.c` file** provided in the template. You should extend and improve this single shell implementation; do **not** create separate copies of `shell.c` for each part.

Note: By default, the provided template `shell.c` tokenizes the input command into whitespace-separated tokens before execution.

```
Shell
➤ ./shell
$ ls -la
DEBUG: found token 'ls'
DEBUG: found token '-la'
$ ls | wc -l
DEBUG: found token 'ls'
DEBUG: found token '|'
DEBUG: found token 'wc'
DEBUG: found token '-l'
$ lx
command not found: lx
$ (on pressing enter)
$
```

Sample output

```

Shell
> gcc shell.c -o shell
> ./shell
$ ls -a
.  ..  shell  shell.c
$ echo hello from our new shell
hello from our new shell
$ ^C
>

```

- c.** Implement the `cd` command. This is a shell-specific command, i.e., it'll be handled separately by the shell process itself instead of calling `exec` in a child process. It must change the current working directory of the shell process itself; you should not spawn a child process to do this. Instead, we've to check for this separately after tokenization and then use the `chdir` system call. You can keep it simple and just accept `cd <directory_name>` as a valid command.

Sample output

```

Shell
> gcc shell.c -o shell
$ ./shell
$ ls -a
.  ..  shell  shell.c
$ pwd
/home/labuser
$ cd /
$ pwd
/

```

- d.** Next, let's implement redirection. For simplicity, the only commands that have redirection are of type `<command> > <file_name>`, `<command> < <file_name>`, `<command> >> <file_name>`.

You can use the `dup2` function for this purpose in a similar fashion to how we used it in Lab2.

Hint: For append redirection (`>>`), open the file using the `O_APPEND` flag (along with `O_WRONLY` | `O_CREAT`) so that all output is written at the end of the file.

Sample output

```
Shell
$ gcc shell.c -o shell
$ ./shell
$ echo some text > output.txt
$ cat output.txt
some text
$ echo more text >> output.txt
$ cat output.txt
some text
more text
$ cat < output.txt
some text
more text
```

- e. Let's add pipes to our shell now. Again, for simplicity, we'll assume the only commands that have pipes are of the format

`<command_1> | <command_2> | ... | <command_n>.`

You can assume an upper bound on n .

The solution will need to create multiple pipes using the `pipe` system call, then connect each command's input to the previous command's output using `dup2`.

Sample output

```
Shell
$ ./shell
$ cat shell.c | grep if | wc -l
86
```

- f. Now comes the interesting part, **background execution!**

A background process is one for which the shell doesn't wait for completion. But it does need to keep track of it and reap it when it's done. We'll assume the only way a background process can be started is via `<command> &` where `<command>` is a simple Linux command, i.e, it doesn't have any shell features like pipes, redirection, etc. We'll also assume an upper bound on the maximum number of background processes.

To ensure all processes are getting reaped no matter foreground or background, the shell needs to keep track of all the background processes it has spawned and keep reaping them, this can happen with a time delay unlike in the case of foreground

processes, as we only check for background processes periodically, for eg. on entering each new command in the shell or in fixed time intervals. You must print “Background process finished: {PID of reaped process}” after reaping the child. Also, since there are now multiple possible child processes of the shell, when we wait for a normal foreground process, it may reap a background one. Be careful and use waitpid to wait for the specific foreground process.

Use long-running commands like `sleep`, and monitor the execution of your process to thoroughly test your shell with both background and foreground processes. One such corner case to test is whether a background process finishing in the middle of a foreground process's execution isn't causing your shell to incorrectly return to the command prompt.

Sample output

```
Shell
> ./shell
$ sleep 4 &
background process started. pid = 319267
$ sleep 10
Background process finished: 319267
```

Submission Guidelines

All submissions via Moodle.

- Name your submissions as: `<rollno>_lab3.tar.gz`
For example, if your roll number is `25b0371`, then the submission will be `2b0371_lab3.tar.gz`
- The tar should contain the following files in the following directory structure:
`<rollno>_lab3/`

```
|__ part1/
|   |__ p1a.c
|   |__ p1b.c
|   |__ p1c.c
|__ part2/
|   |__ p2a.c
|   |__ p2b.c
|   |__ p2c.c
|   |__ p2d.c
|   |__ p2e.c
|__ part3
```


____ p3a.c
____ shell.c

Deadline: **January 22, 2026, 5:00 p.m.**

4. more the merrier v0.2 (optional)

- g.** Currently, when you press Ctrl-C, the whole shell terminates. Extend your shell to handle this so it forwards the SIGINT signal to the foreground process. Background processes must remain unaffected by this. You would need to keep each process in its own process group and selectively send signals to the relevant processes from the shell process. Is it possible to do this just using signal handlers?

Check the correctness of your implementation by starting long-running background processes (like `sleep`) along with a foreground process and sending Ctrl-C.

- h.** Extend your shell to support the `&&` operator for serial foreground execution, where commands are written as `<command_1> && <command_2> ... && <command_n>` and executed sequentially such that each command runs only if the previous one exits successfully (exit status 0). You can assume an upper bound on `n`, and all the commands are simple, i.e., they don't have special features like piping, redirection, etc. Execution must stop immediately on the first failure and return control to the shell prompt. You must execute all commands in the foreground, wait for each specific child using `waitpid`. On encountering Ctrl-C, you must terminate the currently running process, ignore the remaining ones, and return to the command prompt.

Sample output

```
Shell
> ./shell
$ echo first && echo second
first
second
```

- i.** Further extend your shell to support the `&&&` operator for parallel foreground execution, where commands are written as `<command_1> &&& <command_2> ... &&& <command_n>` and all commands are started simultaneously in the foreground. The shell must wait for all spawned processes to complete before returning to the prompt, if all the commands exit with status code 0, it should print "All processes exited"

successfully”; else it should print “There were some errors”. You can assume an upper bound on `n`, and all the commands are simple, i.e., they don’t have special features like piping, redirection, etc. You must track and reap every child process using `waitpid`, ensuring that early completion of one command does not cause the shell to return prematurely. On encountering Ctrl-C, all the foreground running processes must be terminated.

Sample output

```
Shell
> ./shell
$ sleep 2 &&& sleep 4 &&& echo done
done
All processes exited successfully
# (Shell prompt returns after approximately 4 seconds)
```